

Bypassing Web-Application Firewalls by abusing SSL/TLS

Bypassing Web-Application Firewalls by abusing SSL/TLS - Author not stated, 0x09AL Security blog.

- Published: 2018-07-02
- Original: <<https://0x09al.github.io/waf/bypass/ssl/2018/07/02/web-application-firewall-bypass.html>>
- Current location: <<https://blog.pwn.al/waf/bypass/ssl/2018/07/02/web-application-firewall-bypass.html>>
- Preserved from: <https://blog.pwn.al/waf/bypass/ssl/2018/07/02/web-application-firewall-bypass.html> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

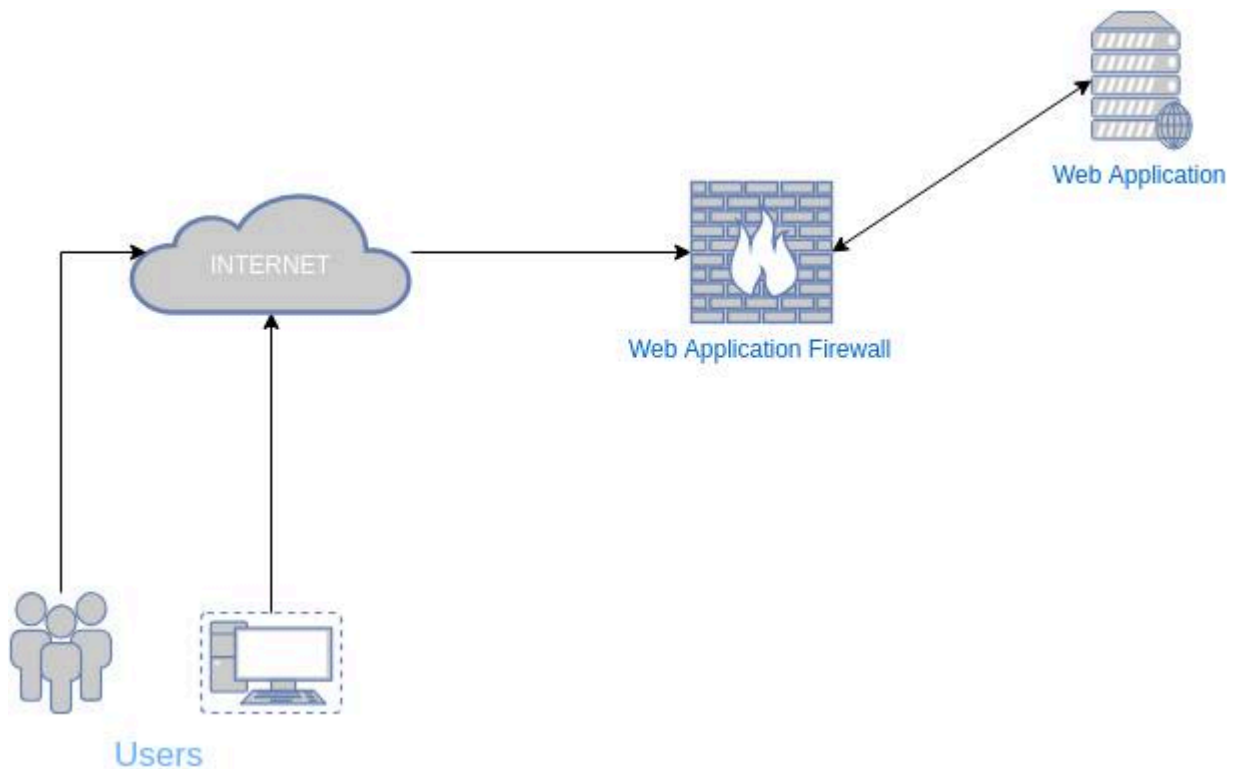
Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

During the latest years Web Security has become a very important topic in the IT Security field. The advantages the web offers resulted in very critical services being developed as web applications. The business requirements for their web applications security has also changed a lot and apart from their good developing standards they add another layer of security. Web Application Firewall's are L7 firewalls which inspect web traffic and "try" to protect from attacks. In this blog post I will explain an interesting bypass vector that I found recently during a deployment audit of a WAF.

The cause of the "vulnerability"

Recently I was assigned to do a deployment test of a WAF in a company. I won't name the company and the product for obvious reasons but the principle stays the same. The architecture of the deployment was something like this:



After I was provided with the required information I started to look for different ways to bypass it. I was given access to the WAF for different tests and apart from other methods I found to bypass it, an interesting one was by abusing SSL Ciphers. The first time I logged in the WAF the `Unsupported SSL Ciphers` alert caught my eye really quick. Once I saw the alert description I started digging more in the documentation of the product and managed to find all the supported SSL ciphers, but before continuing I want to give a quick explanation how an SSL connection works.

The SSL handshake is composed by 3 main phases:

ClientHello/ServerHello Phase.

The handshake begins by the client which sends a ClientHello message. This message contains all the information the server needs, like the various cipher suites and the supported SSL/TLS versions. After receiving the connection the server responds with a ServerHello message which contains similar information that is required by the client. The server also returns what cipher suite and SSL version will be used.

Certificate Exchange

After the connection is initialized the server needs to prove its identity to the client. The server sends the SSL certificate to the client and the client checks if it trust the certificate and continues the connection.

Key Exchange

Now that a secure tunnel is established the server and client exchange a key which will be used for both encryption and decryption of the data.

Attack idea

The idea that popped in my head was : What if I use an "unsupported" SSL Cipher to initialize the connection to the WebServer which supports that cipher, the WAF would not be able to identify the attack because it can't view the data.

So I started digging around and found out the documentation for the WAF vendor, from there I extracted all the SSL Ciphers that were supported. As can be seen below.

SSLv3

```
SSL_RSA_WITH_NULL_MD5
SSL_RSA_WITH_NULL_SHA
SSL_RSA_WITH_RC4_128_MD5
SSL_RSA_WITH_RC4_128_SHA
SSL_RSA_WITH_DES_CBC_SHA
SSL_RSA_WITH_3DES_EDE_CBC_SHA
SSL_RSA_EXPORT_WITH_RC4_40_MD5
SSL_RSA_EXPORT_WITH_DES40_CBC_SHA
```

TLS/1.0-1.2

```
TLS_RSA_WITH_NULL_SHA256
TLS_RSA_WITH_AES_128_CBC_SHA
TLS_RSA_WITH_AES_256_CBC_SHA
TLS_RSA_EXPORT1024_WITH_RC4_56_MD5
TLS_RSA_EXPORT1024_WITH_RC4_56_SHA
TLS_RSA_WITH_AES_128_CBC_SHA256
TLS_RSA_WITH_AES_256_CBC_SHA256
TLS_RSA_WITH_RC4_128_MD5 = { 0x000x04 }
TLS_RSA_WITH_RC4_128_SHA = { 0x000x05 }
TLS_RSA_WITH_DES_CBC_SHA = { 0x000x09 }
```

The next step is to identify the SSL Ciphers that are supported by the webserver.

There are a lot of ways to identify the supported ciphers but I used `ssllscan` because it's easy to install and gives a lot of detailed information.

```
pwn@thinkpad:~$ sudo apt install sslscan
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following NEW packages will be installed:
  sslscan
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 26,7 kB of archives.
After this operation, 81,9 kB of additional disk space will be used.
Get:1 http://al.archive.ubuntu.com/ubuntu bionic/universe amd64 sslscan amd64 1.11.5-
rbsec-1.1 [26,7 kB]
Fetched 26,7 kB in 0s (73,8 kB/s)
Selecting previously unselected package sslscan.
(Reading database ... 177002 files and directories currently installed.)
Preparing to unpack .../sslscan_1.11.5-rbsec-1.1_amd64.deb ...
Unpacking sslscan (1.11.5-rbsec-1.1) ...
Processing triggers for man-db (2.8.3-2) ...
Setting up sslscan (1.11.5-rbsec-1.1) ...
pwn@thinkpad:~$ sslscan http://target/ | grep Accept
```

The command above will list all the supported SSL/TLS versions and ciphers from the webserver.

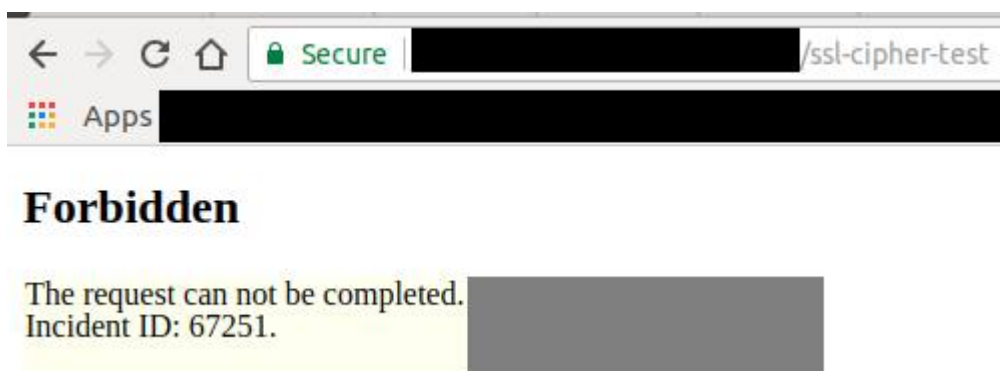
Comparing the result from sslscan and the documentation of the product, I was able to identify some ciphers which were not supported in the Web Application Firewall but supported in the webserver.

Accepted TLSv1 256 bits ECDHE-RSA-AES256-SHA

The cipher supported on the webserver but not on WAF

To test my theory I created a WAF rule which blocked the request if the path of the request was /ssl-cipher-test.

Visiting the path will block the connection successfully.



A quick way to exploit this “bypass” is by specifying the client ciphers, leaving only the one that will bypass the firewall.

You can specify the cipher using `--ciphers` command on curl, in this case I specified ECDHE-RSA-AES256-SHA.

```
pwn@thinkpad:~$ curl --ciphers ECDHE-RSA-AES256-SHA https://waf-test.lab.local/ssl-cipher-test
<html lang=en>
  <title>HELL0 </title>
  <p>Bypass worked</p>
pwn@thinkpad:~$
```

As we see from the response above the Web Application Firewall was bypassed successfully.

Closing remarks

The main plan for me before publishing this blogpost was creating a scanner to scan all the supported ciphers, find one which is supposed to bypass the firewall and then start a proxy listener to forward all the requests with that cipher.

Since that will require a lot of time that I don't have, I decided to release the blogpost and inspire someone to create something based on this research.

References

<https://security.stackexchange.com/questions/67931/why-cant-i-decrypt-ssl-traffic-with-the-clients-private-key-only>

https://www.owasp.org/index.php/TLS_Cipher_String_Cheat_Sheet

[CURL Ciphers Document](#)

Archived from <https://0x09al.github.io/waf/bypass/ssl/2018/07/02/web-application-firewall-bypass.html>.
Rendered offline from the local Markdown copy.